Observing the Village Pond Planning System with OpenTelemetry

CSL559 System Design – OpenTelemetry Assignment (Assignment 1, Phase 3)

LAKSHAY GUPTA

We added OpenTelemetry to our Assignment 1 project, the AI-based Village Pond Planning System, which is deployed at http://10.1.75.51:3305. The FastAPI backend and the React frontend now produce *traces*, *metrics* and *logs*. They send them over OTLP to an OpenTelemetry Collector on a second course container, and the Collector forwards them to OpenObserve, an open-source backend that stores all three signals and shows them in a web UI at http://10.1.75.51:3306. A single trace follows one analysis from the user's click in the browser, through every stage of the backend pipeline, to the three external APIs it calls (AWS elevation tiles, NASA POWER, Open-Meteo). Every screenshot in this report comes from the running deployment. The telemetry answered real questions about the system:

- a 15.4 s request spent 11.1 s waiting behind other users;
- on a warm cache, CPU-bound hydrology, not downloading, is the cost;
- the two rainfall sources win the race almost equally often (30 vs. 26 lookups);
- the observability backend was dropping data, and we found out why.

Additional Key Words and Phrases: OpenTelemetry, Observability, Distributed Tracing, Metrics, Logging, OTLP, OpenObserve

1 WHY THE POND PLANNER NEEDS OBSERVABILITY

The pond planner takes a polygon drawn on a map (or a KML contour file) and returns the best pond sites, their catchments and the water each can collect. One request does the following:

- downloads elevation tiles from AWS;
- queries two rainfall services;
- fills sinks, routes flow (D8) and accumulates it over a grid of up to 250,000 cells;
- ranks candidate sites and delineates each catchment.

All of this runs in one 512 MB, 1 vCPU container that allows a single analysis at a time.

Before this assignment, the only window into the running system was the Uvicorn access log, for example `"POST /api/analyze-area HTTP/1.1" 200`. When a user reported that an analysis "took 15 seconds", that line could not tell us whether the time went on the network, on the CPU, or on waiting behind someone else. As the lecture put it, *in production the only thing you can see is what the system tells you.* The goal of this work is to make the system tell us enough to answer such questions without attaching a debugger.

2 ARCHITECTURE

Figure 1 shows the deployment. It follows the *instrument → collect → store and view* model from the lecture. Each step runs as its own process, and the application code never names the storage backend.

Why two containers. OpenObserve needs about 270–330 MB of memory. The app container already uses about 280 MB at peak (Assignment 1 measurements) out of its 512 MB, so both cannot share it. The course provides four containers, so the Collector and OpenObserve run on a second one (`stu27_sys2`). The two containers reach each other over Docker's internal network.

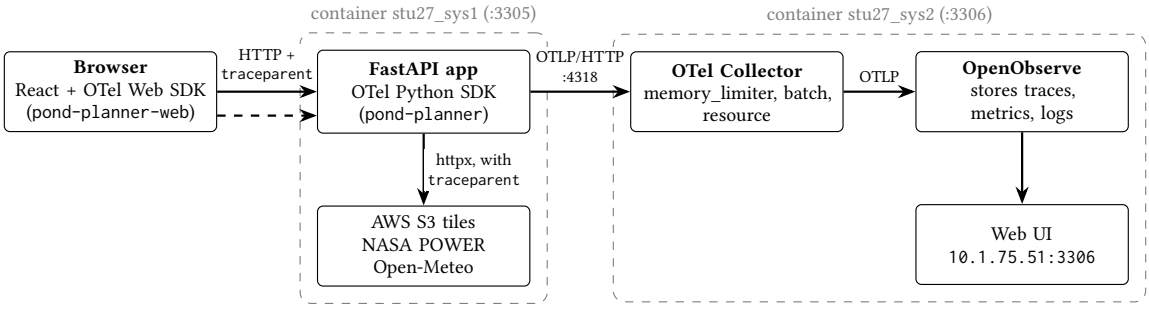


Fig. 1. Deployment. Solid arrows carry requests or telemetry. The dashed arrow shows the browser's spans, which travel via the app's `/otel/v1/traces` endpoint because the browser can only reach port 3305. The Collector listens only on the internal Docker network (172.17.0.107); only the OpenObserve UI is published.

Why a Collector at all. The app could send directly to OpenObserve. Putting a Collector in between has two benefits:

- *Decoupling.* Changing or adding a backend (Jaeger, Prometheus, Grafana) is an edit to one YAML file. The app does not change.
- *Buffering.* The Collector batches data and retries failed sends, so a backend outage does not slow the app. Section 6 shows this mattered.

3 IMPLEMENTATION

3.1 Backend: automatic instrumentation

All set-up lives in one module, `backend/app/telemetry.py`. It is called before the routes are imported, so that the services' HTTP clients are already instrumented when they are created. Four instrumentations need no changes to the application code:

- **FastAPI**: a server span and request-duration metrics for every API call. Health checks and static files are excluded, as are the per-chunk ASGI send/receive spans; a 6.4 MB upload produced 517 of those before we excluded them.
- **httpx**: a client span for every outgoing call (each tile and each rainfall request). It also injects the W3C `traceparent` header into those calls.
- **System metrics**: process memory, CPU, thread count and garbage collection, important on a 512 MB host.
- **Logging**: an OTLP log handler on the root logger, so every existing `logging` call is exported with the trace and span ID of the request that produced it.

3.2 Backend: manual spans

Automatic spans show only the HTTP edges. To see *inside* an analysis we wrapped each pipeline stage in a span (Table 1), using the API from the lecture slides:

```
with tracer.start_as_current_span("terrain.build") as span:
    terrain, dem_info = build_terrain_from_polygon(polygon)
    span.set_attribute("pond.tiles.count", tile_count)
```

Table 1. Manual spans added to the backend

Span	What it covers (key attributes)
`queue.wait`	Waiting for the single processing slot (`pond.queue.wait_s`)
`terrain.build`	Polygon → elevation grid (`pond.area_km2`, `pond.grid.cells`, `pond.tiles.zoom`, `pond.tiles.count`)
`tiles.download`	Fetching missing tiles (`pond.tiles.needed/cached/missing`)
`tiles.decode_and_stitch`	PNG decode and mosaic
`rainfall.lookup`	Cache check and source race (`pond.rainfall.cache_hit`, `pond.rainfall.source`)
`rainfall.nasa_power`, `rainfall.open_meteo`	One span per rainfall source; both run in parallel
`rainfall.wait`	Time the pipeline waited for the background rainfall lookup
`hydrology.fill_sinks_and_d8`	Sink filling and D8 flow direction (`pond.flow.filled_cells`)
`hydrology.flow_accumulation`	Flow accumulation
`hydrology.rank_pond_sites`	Site scoring and selection (`pond.candidates.found`)
`hydrology.catchments_and_volume`	Catchment delineation and runoff volume per site
`contours.generate`, `kml.parse`, `terrain.reconstruct_from_contours`	Contour display, KML parsing, contour-to-grid interpolation

Keeping one trace across threads. The backend runs heavy work in worker threads: the analysis itself, eight parallel tile loads, and the two rainfall sources racing each other. Python's `ThreadPoolExecutor` does not carry the OpenTelemetry context into its threads, so our first version produced a separate, parentless trace for each rainfall lookup. We fixed this with a small wrapper that re-attaches the caller's context in the worker:

```
def in_current_context(func):
    parent = otel_context.get_current()
    def run(*args, **kwargs):
        token = otel_context.attach(parent)
        try:
            return func(*args, **kwargs)
        finally:
            otel_context.detach(token)
    return run
```

With it, one request produces one trace of about 20–25 spans, with correct parent–child links (verified with an in-memory exporter before deployment).

3.3 Backend: custom metrics

Spans describe single requests. Metrics aggregate across all of them cheaply. We added the metrics in Table 2, recorded at the same points as the spans.

Histogram buckets. The SDK's default histogram buckets (0, 5, 10, 25, …) are designed for milliseconds. Our durations are recorded in seconds, so almost every analysis fell into the first "0–5" bucket, and every stage reported the same meaningless p95 of 4.75 s. We noticed this on the first dashboard and set explicit bucket boundaries per metric (for seconds: 0.01 to 120). It is a reminder that a metric is only as good as its aggregation.

3.4 Frontend: browser tracing

The React app uses the OpenTelemetry Web SDK with two instrumentations:

Table 2. Custom metrics (meter pond.planner)

Metric	Type	Attributes / purpose
pond.analysis.count	Counter	mode (area/contour), outcome (success, invalid_area, bad_file_type, ...)
pond.analysis.duration	Histogram (s)	End-to-end analysis time
pond.analysis.stage.duration	Histogram (s)	stage: time per pipeline stage
pond.queue.wait	Histogram (s)	Waiting for the processing slot
pond.analyses.active	UpDownCounter	Analyses running or queued
pond.tiles.requests	Counter	result: cache_hit, downloaded, failed
pond.rainfall.lookups	Counter	source: cache, NASA POWER, Open-Meteo, fallback, user_supplied
pond.candidates.found, pond.grid.cells, pond.area.size, pond.best_site.annual_runoff	Histograms	Size of the work and of the answer

- *document-load*: page-load timing;
- *fetch*: a span for each API call, which also adds a traceparent header to it.

Each analysis is wrapped in a ui.analyze_area (or ui.analyze_contour_file) span. The backend reads the incoming traceparent header, so its server span becomes a child of the browser's span, and the whole request appears as *one* trace across two services. This is the "give every request an ID and pass it on" idea from the lecture, standardised as W3C Trace Context.

The browser cannot reach the Collector, which is internal, so it posts its spans to /otel/v1/traces on the app. The app forwards them unchanged and does not trace this forwarding, to avoid a feedback loop.

3.5 Collector and backend configuration

The Collector (core distribution, v0.162.0) has one pipeline per signal:

- an otlp receiver;
- three processors: memory_limiter (refuses data before the container runs out of memory), batch (one HTTP call per batch rather than per span) and resource (tags data with collector.host);
- two exporters: otlp_http to OpenObserve, and debug, which logs a one-line summary of each batch.

An excerpt of that summary log:

```
2026-10-16 16:50:43 Logs resource logs=2 log records=2
2026-10-16 16:50:48 Traces resource spans=1 spans=13
2026-10-16 16:50:50 Metrics resource metrics=1 metrics=25 data points=536
```

The app is configured only through standard environment variables (OTEL_SERVICE_NAME, OTEL_EXPORTER_OTLP_ENDPOINT). The same code runs locally with no exporter: spans are still created, and trace IDs still appear in the console log.

4 TOOLS, LIBRARIES AND SERVICES

We chose OpenObserve over the common Jaeger + Prometheus + Loki + Grafana stack because it is a single binary that stores and displays traces, metrics and logs together, which fits on one 512 MB container. Because the app speaks plain OTLP, switching to that stack later would only change the Collector's exporter section.

Table 3. Components used

Component	Version	Role
`opentelemetry-api`, `-sdk` (Python)	1.45.1	API and SDK in the backend
`opentelemetry-exporter-otlp-proto-http`	1.45.1	OTLP/HTTP export of traces, metrics, logs
`opentelemetry-instrumentation-fastapi`, `-httpx`	0.66b1	Automatic server and client spans
`opentelemetry-instrumentation-logging`, `-system-metrics`	0.66b1	Log export, process metrics
`@opentelemetry/sdk-trace-web` (+ fetch, document-load)	2.12 / 0.223	Browser tracing
OpenTelemetry Collector (core)	0.162.0	Receive, batch, limit memory, export
OpenObserve	1.0.0	Storage and UI for all three signals
Pond Planner backend	FastAPI 0.141, Python 3.14	The instrumented application
Course containers	Ubuntu 24.04, 512 MB, 1 vCPU	sys1: app; sys2: Collector + OpenObserve

5 RESULTS: WHAT THE SCREENSHOTS SHOW

All screenshots were taken from the running deployment on 16 October 2026. The traffic was a mix of real browser sessions and scripted requests: areas at ten Indian locations (1–25 km^2), one 6.4 MB contour upload, deliberately invalid inputs, and four simultaneous large requests. In about 40 minutes the system recorded 876 spans in 106 traces and 212 log records, plus 81 metric streams.

5.1 The application being observed

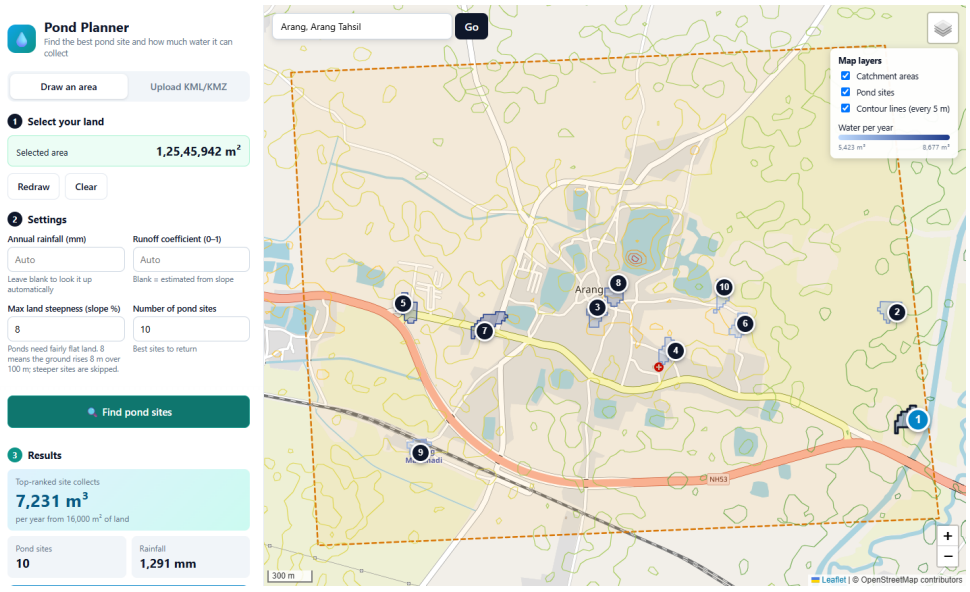


Fig. 2. The pond planner at `10.1.75.51:3305` after a user drew a 12.5 km^2 area around Arang (Chhattisgarh) and clicked *Find pond sites*: ten ranked sites, their catchments and contour lines. This click produced the trace in Figure 3.

5.2 Traces

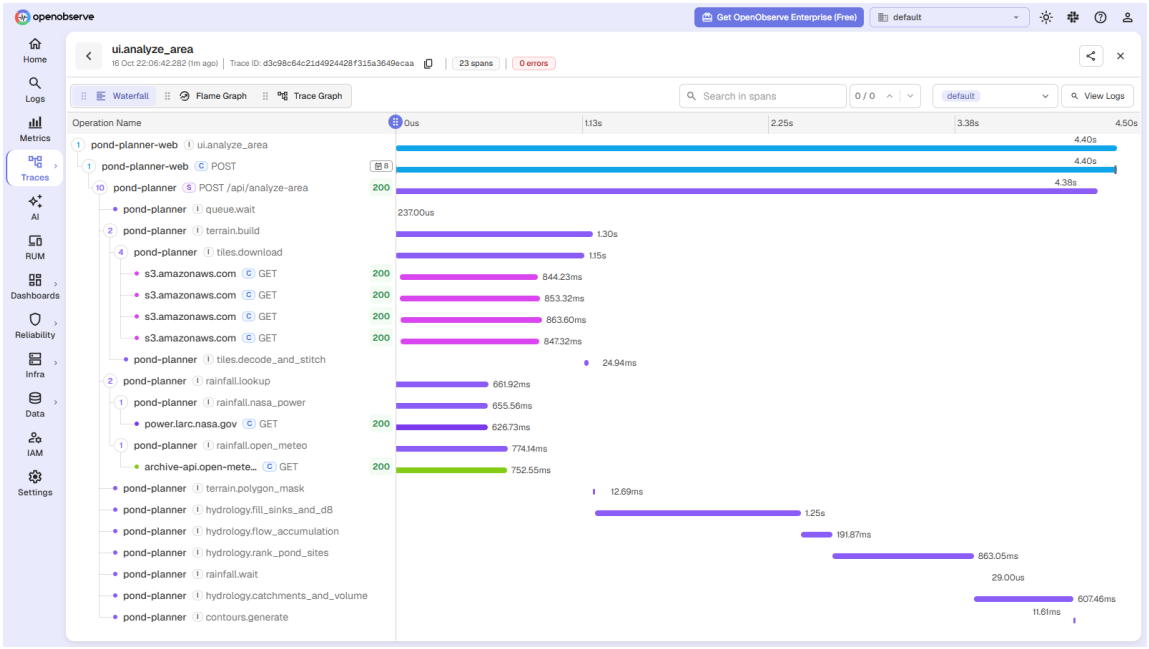


Fig. 3. **One request, end to end (waterfall view).** Trace d3c98c64… has 23 spans from two services: `pond-planner-web` (browser, top two rows) and `pond-planner` (backend). The user waited 4.40 s. Tile download took about 1.15 s (four parallel `GET`s to `s3.amazonaws.com`, 844–864 ms each). The rainfall race, NASA POWER (656 ms) against Open-Meteo (774 ms), ran at the same time as the tile download, so it added nothing to the total. Then came sink filling and D8 (1.25 s), site ranking (0.86 s), and catchments and volumes (0.61 s). `rainfall.wait` is 29 μs: rainfall was ready long before the pipeline needed it, which confirms the overlap design from Assignment 1.

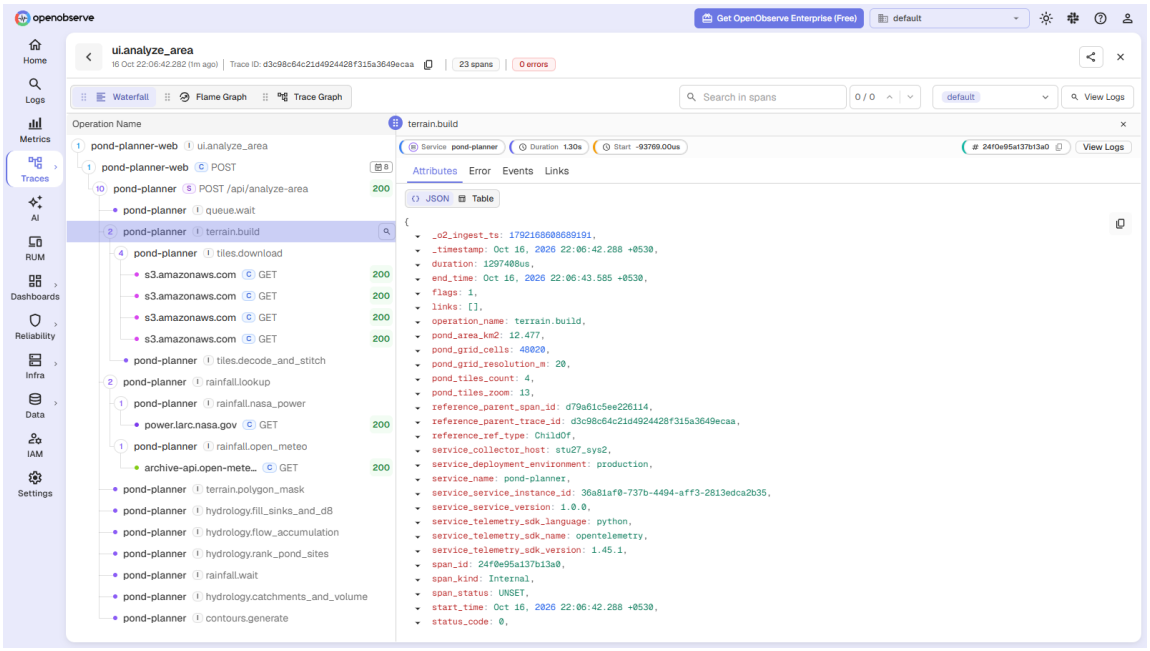


Fig. 4. **Span attributes.** Clicking `terrain.build` shows the business context we attached: the area (12.48 km^2), the grid (48,020 cells at 20 m), tile zoom 13 and four tiles. It also shows resource attributes added automatically or by the Collector: `service_name`, `deployment_environment=production`, SDK language and version, and `collector_host=stu27_sys2`. With these, "slow" requests can be grouped by area or grid size.

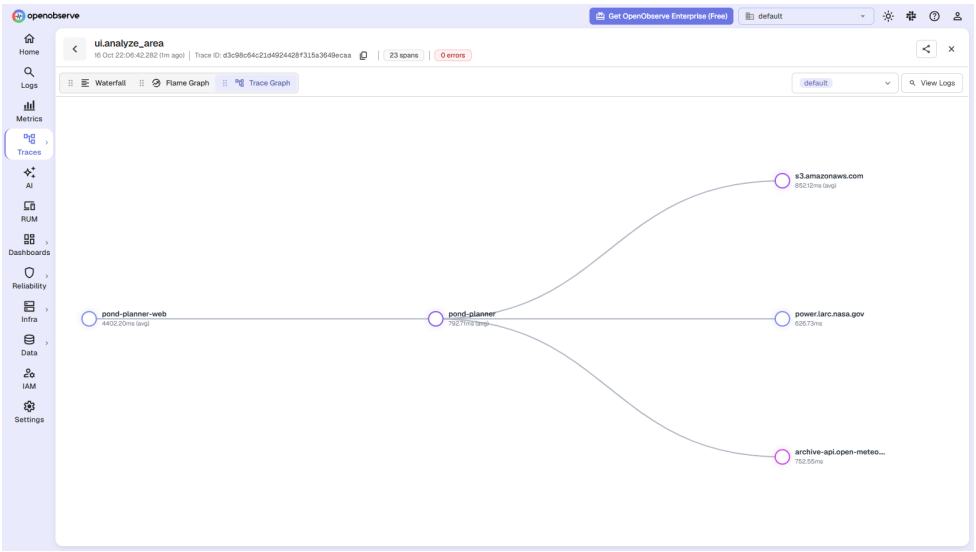


Fig. 5. **Service graph of the same trace.** OpenObserve builds this from parent–child links alone: browser → backend → three external dependencies, with average latencies. We never drew this graph ourselves. It is the system's real dependency map, and it would show a new dependency if one appeared.

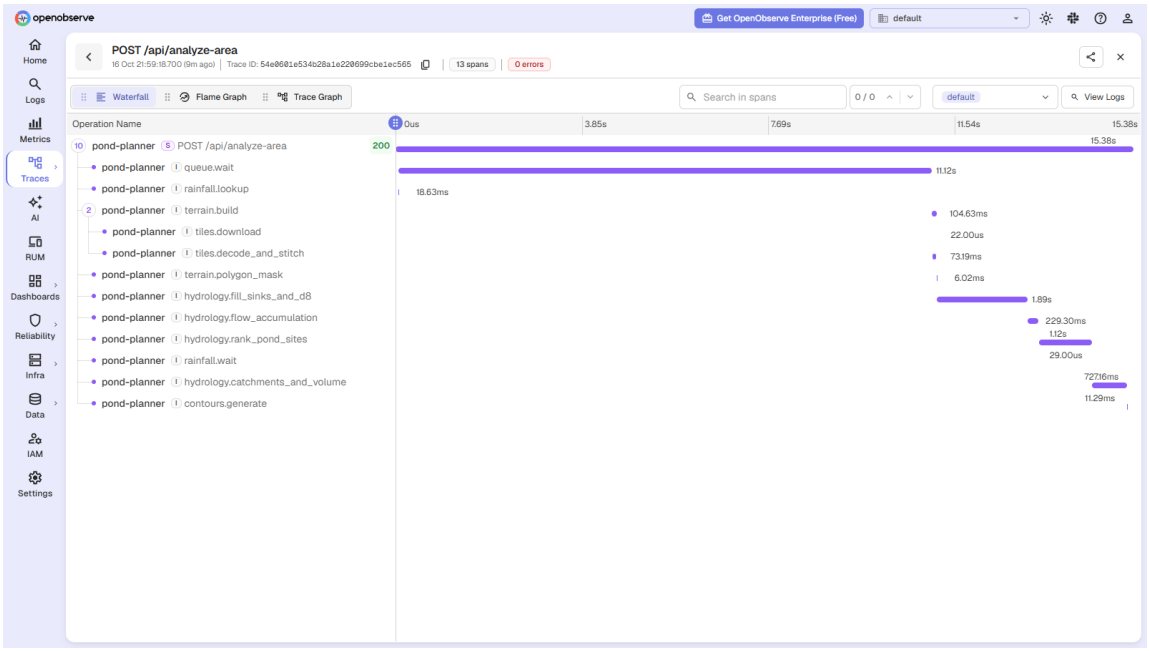


Fig. 6. **Diagnosing a slow request.** Four users submitted 20 km^2 areas at the same moment. The last one waited 15.4 s. Its trace shows that **11.1 s of that was `queue.wait`**: it was queued behind the other three, because the server runs one analysis at a time to stay within 1 vCPU and 512 MB. Its own work took about 4.2 s, the same as an unqueued request. Without the trace, this would look like "the algorithm is slow on big areas". With it, the answer is "the algorithm is fine; we need more capacity or a smaller queue". Tiles were cached, so `tiles.download` took 22 μs.

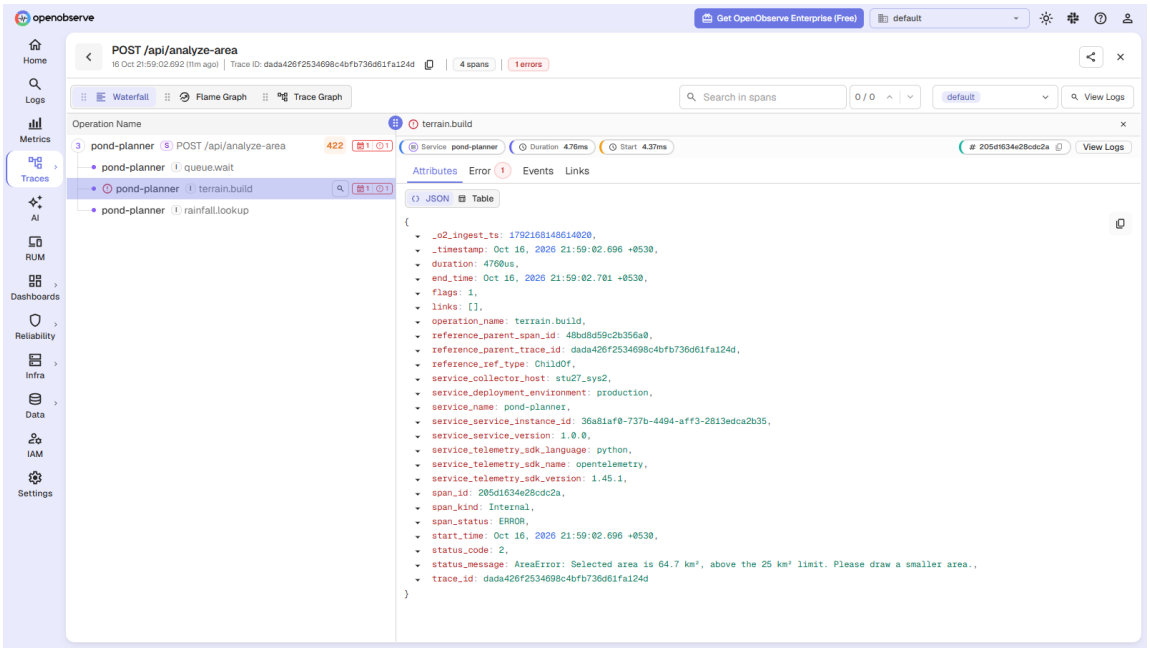


Fig. 7. **An error, located.** A 64.7 km^2 request returned HTTP 422. The trace marks the exact span that failed (`terrain.build`, red icon, span_`status`: `ERROR`). It records the exception message ("Selected area is 64.7 km^2, above the 25 km^2 limit") and shows the request was rejected 4.8 ms into terrain building, before any tile was downloaded. Users are told within milliseconds, and no work is wasted.

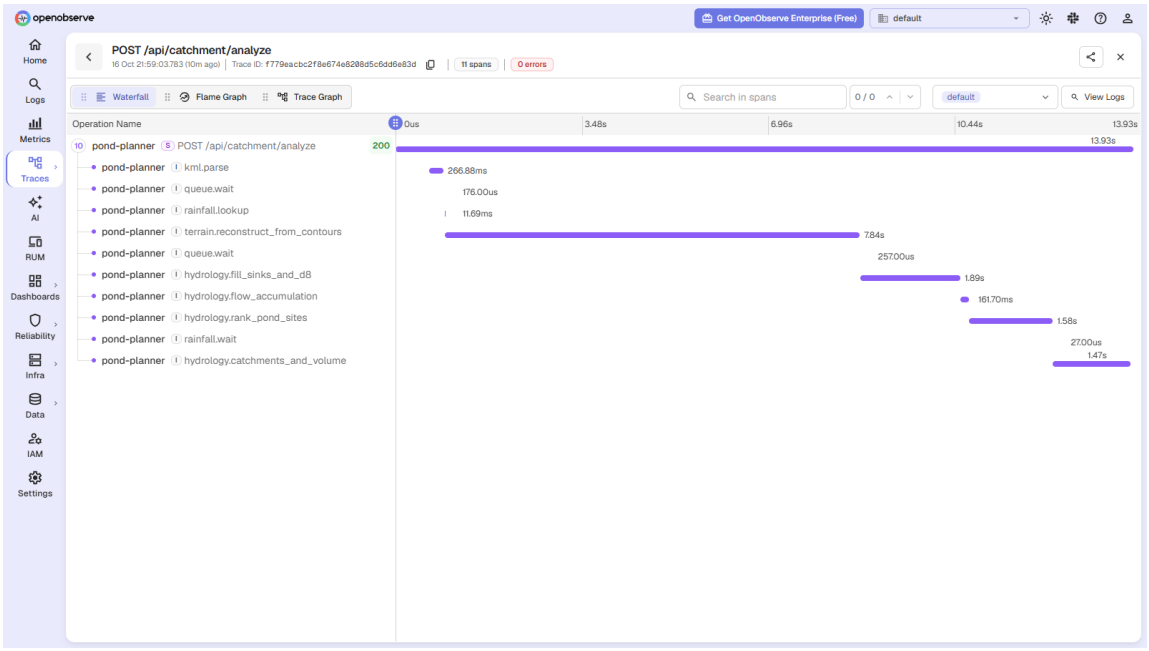


Fig. 8. **The contour-upload path.** A 6.4 MB KML upload took 13.9 s. Parsing took only 0.27 s. `terrain.reconstruct_from_contours` (interpolating contour lines into a grid) took **7.8 s, 56% of the request**, more than all hydrology stages combined (5.1 s). If upload mode needs to be faster, interpolation is the place to optimise. Note the two queue.wait spans: the route takes the processing slot twice.

5.3 Logs

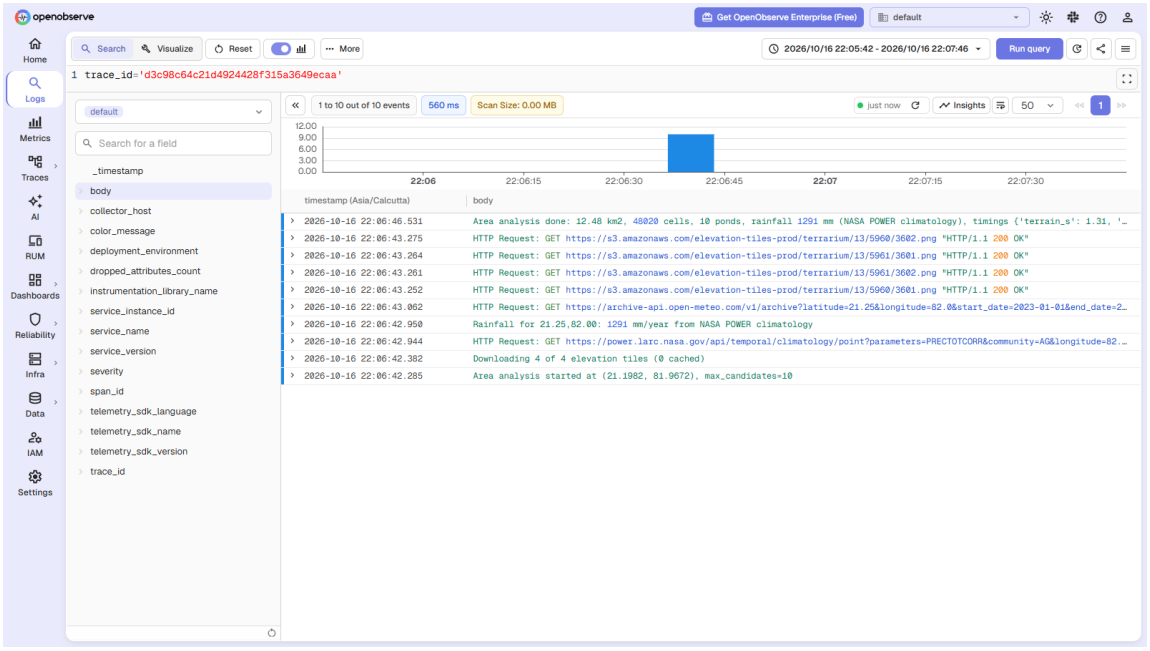


Fig. 9. **Logs correlated with a trace.** The *View Logs* button on the trace in Figure 3 runs the query `trace_id='d3c98c64…'`. It returns exactly the ten log lines this request produced: the start and end of the analysis, "Downloading 4 of 4 elevation tiles (0 cached)", each tile and rainfall HTTP call, and "Rainfall for 21.25,82.00: 1291 mm/year from NASA POWER". Before OpenTelemetry, these lines were interleaved with every other request's lines in `server.log`. Now one ID retrieves them.

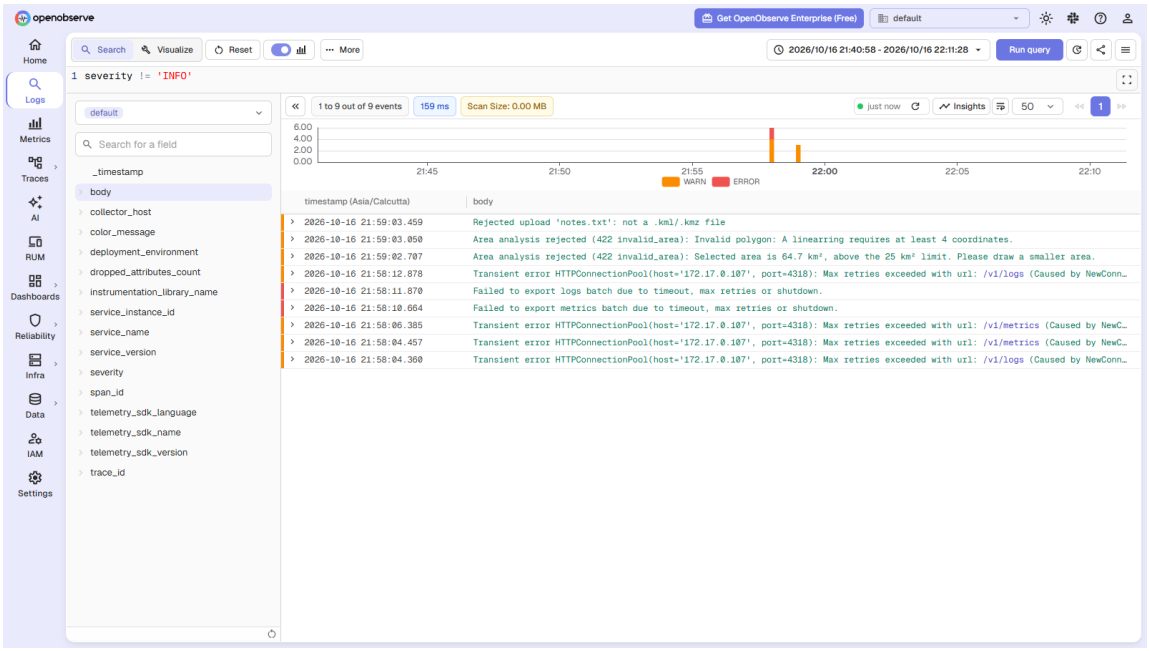


Fig. 10. **All warnings and errors** (`severity != 'INFO'`), coloured by level on the histogram. The three most recent lines are expected rejections: an invalid polygon, an over-limit area and a `.txt` upload. The six lines from 21:58 are more interesting. They are the OpenTelemetry SDK reporting that it *could not reach the Collector* ("Connection refused" on port 4318, then "Failed to export metrics batch") while we restarted the observability container. Telemetry recorded its own outage, and delivered that record once the Collector returned.

5.4 Metrics

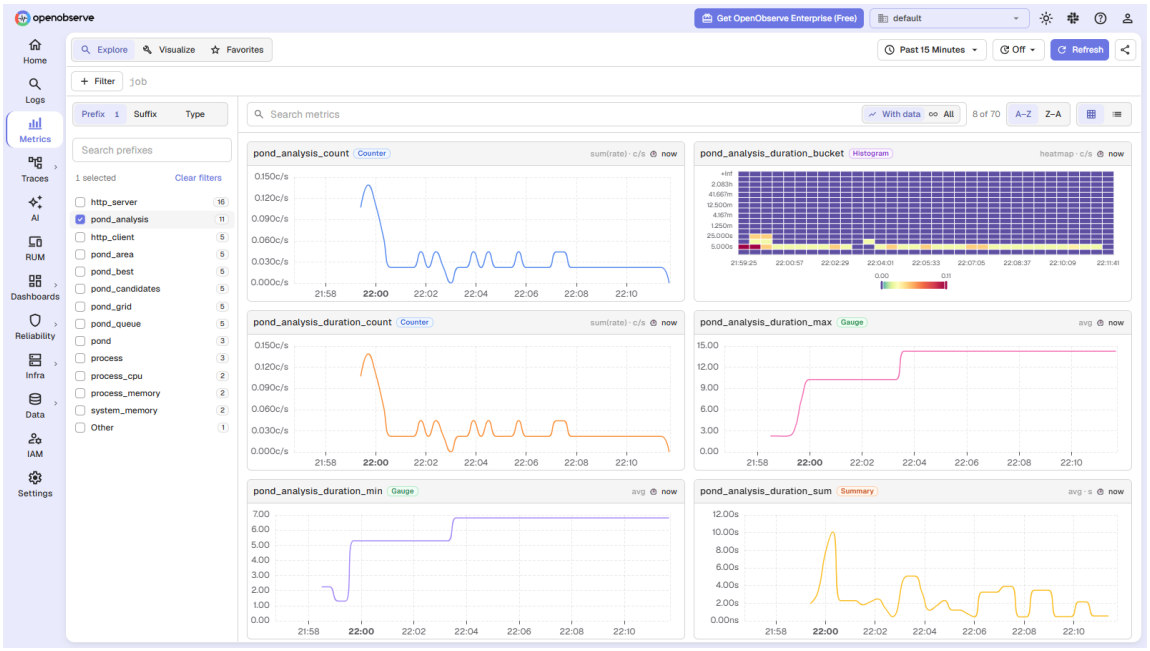


Fig. 11. **Custom analysis metrics** (OpenObserve metrics explorer, prefix pond_analysis). The rate of pond_analysis_count shows the scripted burst at 22:00 and then the slower background load. The duration histogram (_bucket, heat map) shows how analysis times are distributed, not just their average. _max steps up to 15 s when the four simultaneous requests arrived. As the lecture said, a metric tells you *that* something changed, here at 22:00; the trace in Figure 6 tells you *why*.

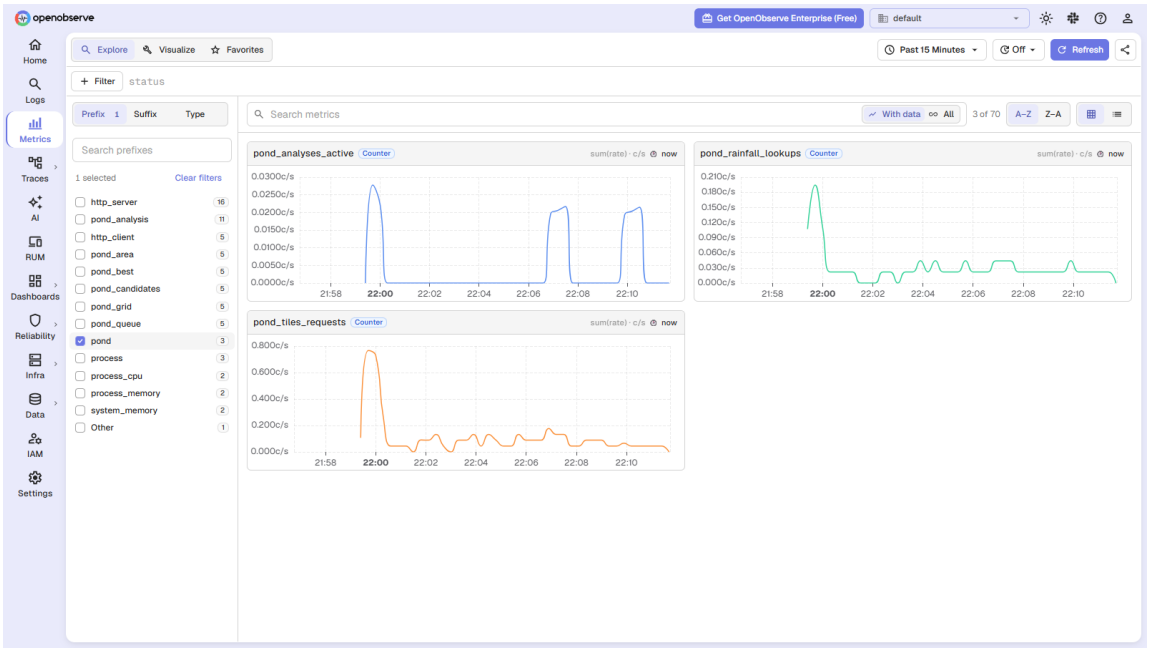


Fig. 12. **Supporting counters:** pond_analyses_active (work in progress or queued), pond_rainfall_lookups (lookups by the source that answered) and pond_tiles_requests (tiles needed per second, mostly cache hits after the first burst). Over the session, 61 tile requests were served from cache and 33 were downloaded. Cold `tiles.download` spans averaged 1.12 s; warm ones took under 1 ms.

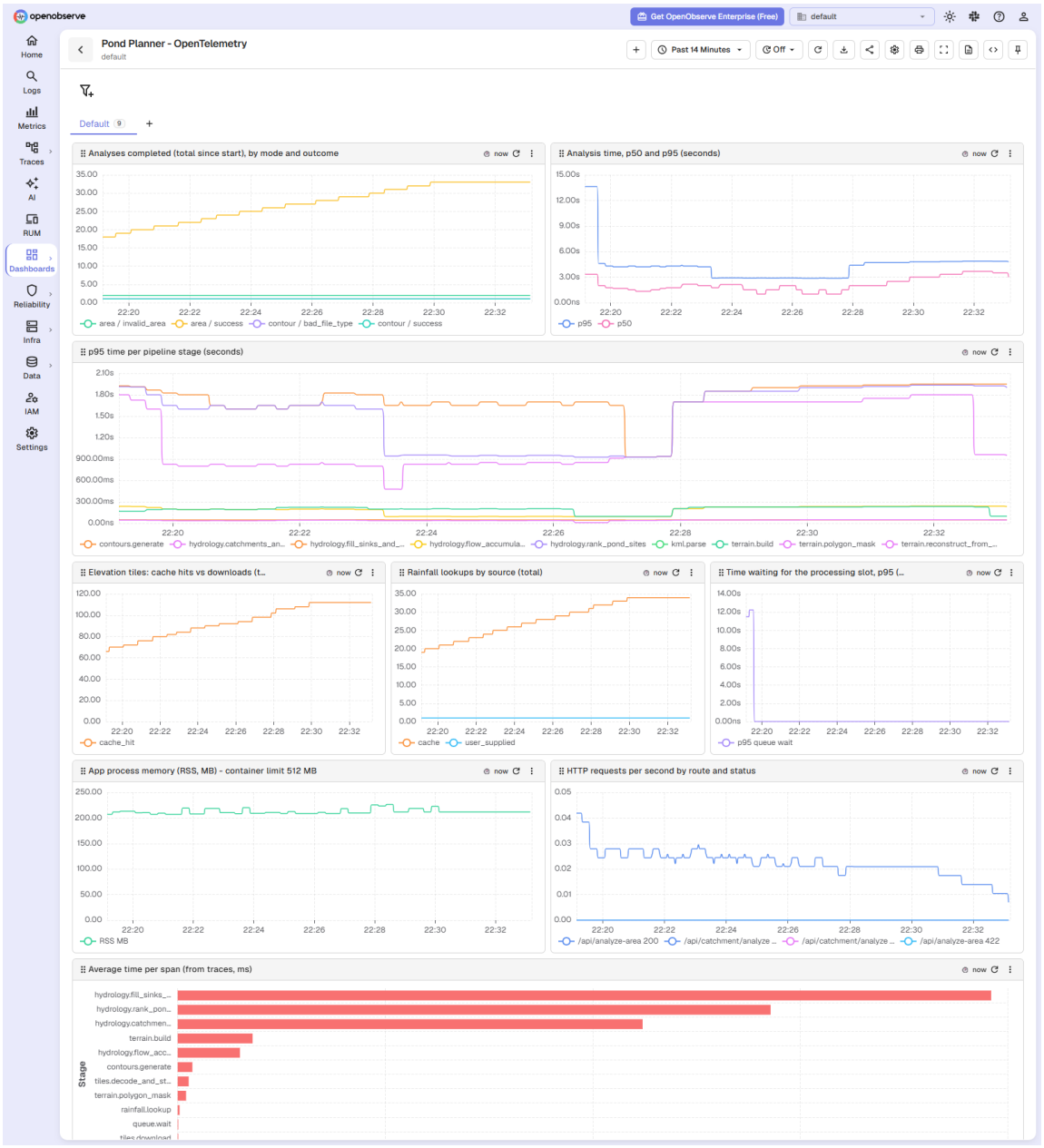


Fig. 13. **Dashboard** built in OpenObserve from PromQL and SQL queries over the same data: analyses by outcome, p50 and p95 analysis time, p95 per pipeline stage, tile cache hits vs. downloads, rainfall source, queue wait, app memory against the 512 MB limit, HTTP rate by route and status, and average span duration taken from the traces. The window is the last 14 minutes of background load, after the histogram fix. Median analysis time is about 2 s and p95 about 4 s. The queue-wait panel shows the four-request burst (p95 12 s) and then zero. App memory holds near 210 MB, well inside the 512 MB limit. Every tile and rainfall lookup in this window was served from cache. Sink filling and D8 is the most expensive stage on average, followed by site ranking.

6 WHAT THE DATA TAUGHT US

Where the time actually goes. Averaged over 56 traced area analyses, the backend stages ranked:

- `hydrology.fill_sinks_and_d8`: 927 ms;
- `tiles.download`: 681 ms, but only on a cold cache, and about 0 ms warm;
- `hydrology.rank_pond_sites`: 620 ms;
- `hydrology.catchments_and_volume`: 492 ms.

In Assignment 1 we assumed downloads dominated and optimised them (async client, caching). The traces confirm that this worked: on a warm cache, the remaining cost is pure CPU work in sink filling and ranking. Those are the next optimisation targets.

Racing two rainfall sources was justified. Every lookup queries NASA POWER and Open-Meteo at once and keeps the first answer. NASA won 30 races and Open-Meteo 26, so neither source is reliably faster. In one local test, NASA's connection was reset and Open-Meteo answered alone. The `rainfall.nasa_power` span recorded the exception, and the request still succeeded. Without the spans, this design decision would have been a guess.

The three signals answer different questions. The lecture's summary held exactly:

- the *metric* (Figure 11) showed that maximum analysis time jumped to 15 s at 22:00;
- the *trace* (Figure 6) showed it was queueing, not computation;
- the *log* ("Analysis waited 11.1 s for the processing slot") recorded it in words, linked by the same trace ID.

None of the three alone would have been enough.

Observability helped debug the observability stack. After the first deployment, data stopped appearing in OpenObserve.

- The Collector's own log showed `Exporting failed. Will retry`, and OpenObserve logged `MemoryTableOverflowError` with HTTP 503.
- The cause: inside the container, both OpenObserve and the Collector see all 120 CPUs of the host machine, not the one vCPU the container is allowed (the Collector logged `NumCPU: 120`). OpenObserve sizes its in-memory buffer buckets and thread pools from the CPU count. Our most likely explanation is that the 32 MB buffer we had configured was spread over so many buckets that each overflowed almost at once.
- The fix: one bucket, a 96 MB buffer, flushing to disk every 60 s, and thread pools sized for one CPU. For the Collector we set `GOMAXPROCS=1`.
- Because the Collector buffers and retries, the app was never slowed, and the delayed data arrived after the fix.

Clock skew is real. The browser's span starts about 110 ms *after* the backend span it caused, which is physically impossible. The laptop's clock is about 110 ms ahead of the server's. Distributed traces combine timestamps from different machines, so durations *within* one service are exact, but offsets *between* services are only as good as clock synchronisation.

Cost. The app process ran at about 150–280 MB RSS with telemetry enabled, within the earlier measured range, and requests took the same time as before (warm 1 km^2: 0.4 s). On the observability host, OpenObserve used 270–330 MB and the Collector 30–85 MB. Batching keeps network overhead low: one export carried 536 metric data points every 15 s.

7 LIMITATIONS

- All traces are kept (no sampling). This is fine at course scale; under real load, a sampler or the Collector's tail sampling would keep only slow and failed traces.
- OpenObserve keeps data on the container's disk with no replication; losing the container loses the history.
- The browser sends its spans through the app, so a user who never reaches the app produces no browser telemetry.
- No alerts are configured yet. An obvious first one is p95 `pond.queue.wait` > 10 s, which would signal that one processing slot is no longer enough.

8 CONCLUSION

OpenTelemetry turned the pond planner from a black box with an access log into a system we can question. Every request now has one ID that links:

- the user's click;
- every pipeline stage and external call;
- its log lines;
- the aggregate metrics it contributes to.

The instrumentation is vendor-neutral, which is what the lecture called "instrument once, send anywhere": the code contains no reference to OpenObserve, and the backend can be changed in the Collector's configuration alone. The questions it answered (queueing vs. computation, cold vs. warm cost, which rainfall source wins, why data was being dropped) were real ones, and their answers changed what we would do next.

A REPRODUCING THE SET-UP

```
# Observability container (stu27_sys2): binaries + config in ~/observability
./start.sh # OpenObserve on :3000 (published as :3306), Collector on :4318

# App container (stu27_sys1): ~/pond
export OTEL_SERVICE_NAME=pond-planner
export OTEL_EXPORTER_OTLP_ENDPOINT=http://172.17.0.107:4318
venv/bin/uvicorn app.main:app --app-dir backend --host 0.0.0.0 --port 3000
```

Source: `backend/app/telemetry.py` (set-up and metrics), `frontend/src/telemetry.js` (browser), `backend/app/api/routes/otel_proxy.py` (browser span forwarding), `observability/otelcol-config.yaml`, `observability/start.sh`, `observability/run-app.sh`.

USE OF AI ASSISTANCE

Claude (Anthropic) was used to help prepare this report and, where necessary, to understand the concepts and tools involved.